HP2L48 Actual Examination Marking Scheme

Unit: Software Development: Object Oriented Programming (SCQF Level 8)
Assessment Title: Sokoban — Simplified Version (Final Examination)
Total Marks: 100
Stages: Stage 1 (Implementation — Outcomes 1 & 2) + Stage 2 (Testing — Outcome 3)

Grade Boundaries & Override Rule

Grade	Mark Range	Description
S (Satisfactory)	70–100	All core requirements met to a good or excellent standard.
RD (Resubmit with Direction)	40–69	Significant gaps or weaknesses. Specific guidance will be provided for resubmission.
RA (Resubmit Attempt)	1–39	Fundamental weaknesses or major omissions. Substantial rework needed.
NYA (Not Yet Attempted)	0	No meaningful submission.

Override Rule

Even if a candidate scores 70 or above, they **cannot** be awarded **S** if **any** of the following are completely missing:

1. No Stage 1 code submitted.
2. No Test Plan submitted.
3. No Test Log submitted.
4. Fewer than **4 classes** implemented in the submitted code.
5. No **inheritance** relationship demonstrated anywhere in the code.

6. No **encapsulation** — defined as: the majority of class-level instance variables are declared `public` without justification (variables declared `private` or `protected` for inheritance purposes are considered properly encapsulated).
7. If the difficulty of moving one level to the next level does not raise correspondingly(like the number of crates or the path difficulty or the possible solution counts et), another 5 points of decreasing will be made.

In such cases, the **maximum possible grade is RD**.

Outcome 1 — OOP Design Quality (30 marks)

1.1 Class Design & Cohesion (8 marks)

Criterion	Marks	Objective Evidence (Yes / No)
Each class has a single, clear responsibility	3	**3 marks:** No class contains unrelated functionality (e.g. no class handles both game logic AND graphical drawing). **2 marks:** One class has mixed responsibilities but the rest are well-defined. **1 mark:** Most classes have mixed responsibilities. **0 marks:** All logic is concentrated in one or two "god classes".
No "god class" containing all program logic	3	**3 marks:** Logic is distributed across at least 4 classes with distinct roles. **1 mark:** One class contains more than 60% of all methods. **0 marks:** One class contains nearly all methods and other classes are minimal.
Controller/Logic classes are separate from View/UI classes	2	**2 marks:** At least one class manages logic and a different class manages drawing/UI. **0 marks:** Drawing and logic are combined in the same class.

1.2 Encapsulation & Information Hiding (7 marks)

Criterion	Marks	Objective Evidence (Yes / No)
All class-level instance variables are `private` or `protected` (where justified for inheritance)	4	**4 marks:** Zero `public` instance variables across all classes. **3 marks:** 1–2 `public` instance variables. **2 marks:** 3–4 `public` instance variables. **1 mark:** More than half of instance variables are `public`. **0 marks:** All or nearly all instance variables are `public`.
Data is accessed through methods (getters/setters or other methods), not by direct field access from unrelated classes	3	**3 marks:** All inter-class data access uses method calls. **2 marks:** 1–2 instances of direct field access from unrelated classes. **0 marks:** Frequent direct access to fields of other classes (content coupling present).

1.3 Inheritance & Polymorphism (8 marks)

Criterion	Marks	Objective Evidence (Yes / No)
At least one meaningful inheritance hierarchy exists (e.g. Wall, Floor, Crate extend a common superclass)	3	**3 marks:** At least one superclass with 2+ subclasses. Each subclass adds specific behaviour or attributes. **2 marks:** One superclass with exactly 1 subclass. **0 marks:** No inheritance used at all.
Subclass constructors correctly call `super()`	2	**2 marks:** All subclass constructors include a `super()` call. **1 mark:** Some subclass constructors are missing `super()`. **0 marks:** No `super()` calls present.
At least one method is overridden in a subclass (with `@Override` annotation)	2	**2 marks:** At least one method is overridden in a subclass with `@Override`. **1 mark:** Method is overridden but `@Override` annotation is missing. **0 marks:** No method overriding demonstrated.
Polymorphic behaviour is demonstrated (e.g. superclass	1	**1 mark:** Code uses a superclass reference (e.g. array or ArrayList of superclass type) that stores

Criterion	Marks	Objective Evidence (Yes / No)
reference used to call overridden methods)		subclass objects and calls overridden methods. **0 marks:** No polymorphic usage visible.

1.4 Relationships — Association, Aggregation, One-to-Many (4 marks)

Criterion	Marks	Objective Evidence (Yes / No)
At least one clear one-to-many relationship is implemented	4	**4 marks:** A class contains an array or ArrayList of another class (e.g. Map contains MapElement[][], or Game contains ArrayList). **0 marks:** No one-to-many relationship exists.

1.5 Coupling (3 marks)

Criterion	Marks	Objective Evidence (Yes / No)
Classes communicate primarily through parameter passing (data coupling), not through public static variables or direct field access	3	**3 marks:** No public static mutable variables used for inter-class communication. All data passed via method parameters. **2 marks:** 1 public static variable used for communication. **1 mark:** 2+ public static variables used. **0 marks:** Extensive use of public static variables for inter-class communication (common coupling).

Outcome 2 — Implementation (40 marks)

2.1 Working Solution — Feature Breakdown (12 marks)

Criterion	Marks	Objective Evidence (Yes / No)
Player can move in at least 4 directions using keyboard input	3	**3 marks:** Player moves correctly in all 4 directions (up/down/left/right). **2 marks:** Player moves in 2–3 directions. **1 mark:** Player moves in only 1 direction. **0 marks:** Player does not move at all.
Player can push a crate/box to an adjacent empty cell	3	**3 marks:** Crate moves correctly when pushed by the player. **2 marks:** Crate moves but has minor issues (e.g. wrong direction occasionally). **1 mark:** Crate sometimes moves but logic is largely broken. **0 marks:** Crates cannot be pushed at all.
Crate/box changes visual state when placed on a target/diamond	3	**3 marks:** Crate visually changes (e.g. colour, image) when on a target. **2 marks:** Visual change is present but inconsistent. **0 marks:** No visual feedback when crate reaches target.
Level completion is detected and displayed (all crates on targets → win message or next level)	3	**3 marks:** Win condition is correctly detected and a message or level transition occurs. **2 marks:** Win is detected but message is unclear or level transition fails. **1 mark:** Win detection logic exists but is partially incorrect. **0 marks:** No win detection at all.

2.2 SQA Mandatory OOP Requirements (10 marks)

Criterion	Marks	Objective Evidence (Yes / No)
At least 2 different data structures are used (e.g. 2D array AND ArrayList, or 1D array AND ArrayList)	2	**2 marks:** Two or more distinct data structure types are declared and used. **1 mark:** Only one data structure type is used throughout. **0 marks:** No structured data types used.

Criterion	Marks	Objective Evidence (Yes / No)
Program contains at least 4 classes	2	**2 marks:** 4 or more classes exist. **1 mark:** Exactly 3 classes. **0 marks:** 2 or fewer classes.
At least 3 objects are instantiated from classes (using `new` keyword)	2	**2 marks:** 3 or more objects created from different or same classes. **1 mark:** 1–2 objects created. **0 marks:** No objects instantiated.
At least 1 overloaded method or constructor exists	2	**2 marks:** A class contains 2+ methods/constructors with the same name but different parameter lists. **0 marks:** No overloading demonstrated.
Parameters are passed correctly between objects (at least 3 distinct examples)	2	**2 marks:** 3+ examples of parameter passing between different objects. **1 mark:** 1–2 examples. **0 marks:** No parameter passing between objects.

2.3 Code Documentation & Style (8 marks)

Criterion	Marks	Objective Evidence (Yes / No)
Every class has a class-level comment explaining its purpose	2	**2 marks:** All or nearly all classes have a class-level comment (e.g. `/** This class represents... */` or `// This class handles...`). **1 mark:** More than half of classes are missing class-level comments. **0 marks:** No class-level comments at all.
Every method has a brief comment explaining its purpose	2	**2 marks:** All or nearly all methods have a comment above them. **1 mark:** More than half of methods are missing comments. **0 marks:** No method-level comments at all.
Naming conventions are followed consistently (camelCase for variables/methods, PascalCase for classes)	2	**2 marks:** Naming is consistent throughout. **1 mark:** 3–5 naming inconsistencies. **0 marks:** Naming is random or consistently violates conventions.

Criterion	Marks	Objective Evidence (Yes / No)
Code indentation is consistent and readable	2	**2 marks:** Code is consistently indented and easy to read. **1 mark:** Indentation is inconsistent in several places. **0 marks:** Code is not indented and very difficult to read.

2.4 Error Handling & Robustness (6 marks)

Criterion	Marks	Objective Evidence (Yes / No)
Player cannot move into walls or out of bounds (movement is blocked gracefully, no crash)	3	**3 marks:** Player is prevented from moving into walls/boundaries with no crash. **2 marks:** Player is blocked but error handling is inconsistent (e.g. works for some directions only). **1 mark:** Player can sometimes move into walls. **0 marks:** Program crashes or allows movement through walls.
Crate cannot be pushed into walls, out of bounds, or into another crate (push is blocked gracefully, no crash)	3	**3 marks:** Invalid push is prevented with no crash. **2 marks:** Push is blocked but handling is inconsistent. **1 mark:** Push logic partially works but allows some invalid pushes. **0 marks:** Program crashes on invalid push or crates can be pushed through walls.

2.5 Use of Standard Libraries (4 marks)

Criterion	Marks	Objective Evidence (Yes / No)
Appropriate use of Java standard UI classes (e.g. JFrame, JPanel, JLabel, JButton, Graphics)	2	**2 marks:** At least 2 different Swing/AWT classes are used correctly. **1 mark:** Only 1 Swing/AWT class is used. **0 marks:** No UI classes used (console-only program).
Appropriate use of Java standard event classes (e.g. KeyListener,	2	**2 marks:** At least 1 event listener class is used correctly to handle user input. **0 marks:** No event listeners used.

Criterion	Marks	Objective Evidence (Yes / No)
KeyAdapter, ActionListener, MouseAdapter)		

Outcome 3 — Testing (30 marks)

3.1 Test Plan Quality (10 marks)

Criterion	Marks	Objective Evidence (Yes / No)
Test Plan is submitted separately or clearly identifiable before testing (Expected Results column is filled in, Actual Results column is blank or marked "pending")	2	**2 marks:** Expected Results are clearly written before testing; Actual Results are not pre-filled. **0 marks:** Both Expected and Actual are filled in simultaneously (post-hoc planning).
Normal cases are included (e.g. move player, push crate, complete level)	3	**3 marks:** At least 4 distinct Normal test cases covering different features. **2 marks:** 2–3 Normal test cases. **1 mark:** Only 1 Normal test case. **0 marks:** No Normal test cases.
Boundary cases are included (e.g. player against wall, crate against wall, crate pushed onto target)	3	**3 marks:** At least 2 distinct Boundary test cases. **2 marks:** 1 Boundary test case. **0 marks:** No Boundary test cases.
Error/Exception cases are included (e.g. push crate into wall, push crate into another crate, move into wall)	2	**2 marks:** At least 1 Error/Exception test case. **0 marks:** No Error/Exception test cases.

3.2 Test Log Specificity & Format (10 marks)

Criterion	Marks	Objective Evidence (Yes / No)
Actual Results are described specifically (e.g. "Crate changed to red colour", "Moves counter increased to 5") — NOT just "Worked" or "OK"	4	**4 marks:** All or nearly all Actual Results contain specific descriptions. **3 marks:** Most Actual Results are specific; a few say only "Pass" or "Worked". **2 marks:** Half of Actual Results are specific. **0 marks:** All or nearly all Actual Results say only "Worked" or "Pass" with no detail.
Pass/Fail is clearly marked for every test case	3	**3 marks:** Every test case has a clear Pass or Fail label. **2 marks:** 1–2 test cases are missing Pass/Fail labels. **0 marks:** No Pass/Fail labels present.
All planned test cases have been executed and recorded	3	**3 marks:** Every test case in the Test Plan has a corresponding Actual Result(Screenshot included). **2 marks:** 1–2 test cases are missing Actual Results or some screenshots are missing. **0 marks:** More than 2 of test cases are missing Actual Results.

3.3 Bug Handling & Retesting (6 marks)

Criterion	Marks	Objective Evidence (Yes / No)
At least one genuine Fail is recorded with a description of what actually happened	3	**3 marks:** At least 1 test case is marked Fail with a specific description of the incorrect behaviour. **1 mark:** A Fail is recorded but the description is vague (e.g. "Didn't work"). **0 marks:** No Fail is recorded at all. (Note: A program with no Fails is acceptable ONLY if the program is genuinely fully functional and robust. If obvious bugs exist but all tests show Pass, award 0 marks here.)
The Fail is analysed (cause identified), code is fixed, and a Retest is documented with the result	3	**3 marks:** A code fix is described, and a Retest row shows the same test now Passes. **2 marks:** A fix is described but no Retest is documented. **1 mark:** A Retest exists but no fix description is provided. **0 marks:** No fix or Retest documented. Note: A candidate who records zero Failures

Criterion	Marks	Objective Evidence (Yes / No)
		must include an explicit declaration that no bugs were detected (see Integrity Rule below).

3.4 Evaluation (4 marks)

Criterion	Marks	Objective Evidence (Yes / No)
A brief written evaluation summarises the testing process, bugs found, and overall assessment of program robustness	4	**4 marks:** Evaluation is present, references specific tests/bugs, and provides a clear overall assessment. **3 marks:** Evaluation is present but only partially addresses the points above. **2 marks:** Evaluation is very brief or generic (e.g. "Testing went well"). **0 marks:** No evaluation provided.

Integrity Rule for Testing Evidence

A candidate who does not record any test failures must include a clear and explicit statement declaring that all tests passed and no bugs or errors were detected during testing. If such a statement is provided, the candidate accepts full responsibility for its accuracy. If, upon inspection of the submitted code, the assessor discovers any bug or error — particularly one that should have been identified by the executed test cases or that affects basic program functionality — the candidate will lose all marks in Sections 3.3 and 3.4, as the testing evidence is deemed unreliable. If no such statement is provided and no failures are documented, the candidate will also lose all marks in Sections 3.3 and 3.4, as the absence of both a failure record and a declaration renders the testing process unverifiable.

Summary Mark Sheet

Outcome 1 — OOP Design Quality

Section	Available	Awarded
1.1 Class Design & Cohesion	8	

Section	Available	Awarded
1.2 Encapsulation & Information Hiding	7	
1.3 Inheritance & Polymorphism	8	
1.4 Relationships	4	
1.5 Coupling	3	
Outcome 1 Total	**30**	

Outcome 2 — Implementation

Section	Available	Awarded
2.1 Working Solution	12	
2.2 SQA Mandatory OOP Requirements	10	
2.3 Code Documentation & Style	8	
2.4 Error Handling & Robustness	6	
2.5 Use of Standard Libraries	4	
Outcome 2 Total	**40**	

Outcome 3 — Testing

Section	Available	Awarded
3.1 Test Plan Quality	10	
3.2 Test Log Specificity & Format	10	
3.3 Bug Handling & Retesting	6	
3.4 Evaluation	4	
Outcome 3 Total	**30**	

Final Grade

Total Mark	/100
Grade	S / RD / RA